

Main Plan

Prokoi: Project Management System v1.0

Goal: Build a lightweight, Jira-inspired task tracker focused strictly on core organizational needs to fulfill the lab requirements and secure an A grade.

Core Architecture & Access

- Organizations serve as the root level for grouping all system data.
- Teams & Projects provide dedicated spaces to separate different working groups.
- Access Management handles assigning specific team and project members to restrict task visibility.

Issue Tracking & Task Management

- Issue Creation allows users to make simple tasks or bugs with titles and descriptions.
- Issue Lists display a clear, centralized view of all issues belonging to a specific project.
- Basic State Tracking moves issues through a minimal lifecycle (To-Do, In Progress, Done).
- Task assignment explicitly links an individual team member to a specific issue to establish clear ownership.

Milestones & Progress Tracking (Teacher Requirements)

- Milestone tracking groups relevant tasks together under a single major deliverable phase.
- Dynamic Progress Bars visually update to reflect the percentage of completed tasks within a milestone.
- A Progress/Comments logging system allows team members to document specific delays, such as noting the exact number of days lost due to sickness.

Workspace & Resource Centralization (Teacher Requirements)

- A dedicated Repo Link section stores the central repository, team meeting links, and cloud storage URLs for easy data access.

- The system architecture is designed to support future additions like issue comments, attachments, and email notifications.

Phase 1

Here is the complete and updated Phase 1 breakdown, incorporating the **teams** layer to perfectly match the **Organizations -> Teams -> Projects -> Issues** hierarchy.

Phase 1: Core Foundation Features

Focus strictly on establishing the workspace hierarchy and access control.

- **User Authentication:** Simple registration and login for all system users.
- **Organization Management:** The ability to create root-level containers for grouping all data.
- **Team Management:** Creating specific teams (e.g., "Backend", "UI/UX") within an organization.
- **Project Spaces:** Creating isolated project environments assigned to a specific team.
- **Resource Links:** A project settings form to save the Git repository URL, storage link, and meeting link.
- **Access Management:** Assigning users to specific organizations and granting granular roles within specific teams.

Database Architecture

Designed for PostgreSQL using strict foreign keys to keep your raw SQL **JOIN** operations fast and predictable.

1. **users** Table

Stores all individuals accessing the system.

Column	Type	Constraints	Description
id	UUID/SERIAL	PRIMARY KEY	Unique identifier.
name	VARCHAR(100)	NOT NULL	User's full name.

email	VARCHAR(150)	UNIQUE, NOT NULL	Used for login.
password_hash	VARCHAR(255)	NOT NULL	Securely hashed password.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time.

2. organizations Table

The root-level container.

Column	Type	Constraints	Description
id	UUID/SERIAL	PRIMARY KEY	Unique identifier.
name	VARCHAR(100)	NOT NULL	Organization name.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time.

3. organization_members Table (Junction)

Maps users to organizations for top-level access.

Column	Type	Constraints	Description
org_id	UUID/INT	FOREIGN KEY (organizations.id)	The organization.

<code>user_id</code>	UUID/INT	FOREIGN KEY (<code>users.id</code>)	The user.
<code>role</code>	VARCHAR(50)	NOT NULL	E.g., 'ADMIN', 'MEMBER'.

4. `teams` Table

Groups specific units together beneath the organization umbrella.

Column	Type	Constraints	Description
<code>id</code>	UUID/SERIAL	PRIMARY KEY	Unique identifier.
<code>org_id</code>	UUID/INT	FOREIGN KEY (<code>organizations.id</code>)	Links the team to an org.
<code>name</code>	VARCHAR(100)	NOT NULL	Team name.
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time.

5. `team_members` Table (Junction)

Handles granular access control at the team level.

Column	Type	Constraints	Description
<code>team_id</code>	UUID/INT	FOREIGN KEY (<code>teams.id</code>)	The specific team.

<code>user_id</code>	UUID/INT	FOREIGN KEY (<code>users.id</code>)	The user being assigned.
<code>role</code>	VARCHAR(50)	NOT NULL	E.g., 'LEAD', 'MEMBER'.

6. `projects` Table

Dedicated spaces assigned to a specific team.

Column	Type	Constraints	Description
<code>id</code>	UUID/SERIAL	PRIMARY KEY	Unique identifier.
<code>team_id</code>	UUID/INT	FOREIGN KEY (<code>teams.id</code>)	Links project to a team.
<code>name</code>	VARCHAR(150)	NOT NULL	Project name.
<code>repo_link</code>	TEXT	NULLABLE	GitHub/GitLab link.
<code>meeting_link</code>	TEXT	NULLABLE	Zoom/Meet link.
<code>storage_link</code>	TEXT	NULLABLE	Drive/Figma link.
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time.

Do you want to write the raw SQL `CREATE TABLE` script to execute in your PostgreSQL database, or update the Phase 2 database tables to match this new hierarchy?

Api Design

Here is the updated REST API design for Phase 1. This reflects the complete **Organizations -> Teams -> Projects** hierarchy, making it clean and intuitive for your React frontend to navigate.

1. Users & Authentication (/api/users)

Handles getting your team registered and logged in.

Method	Endpoint	Request Payload	Description
POST	/api/users/register	{ name, email, password }	Hashes the password and creates a new user.
POST	/api/users/login	{ email, password }	Authenticates the user and returns a token (e.g., JWT).
GET	/api/users/me	None (Requires Auth Token)	Returns the logged-in user's details.

2. Organizations (/api/organizations)

The root level of your API.

Method	Endpoint	Request Payload	Description
POST	/api/organizations	{ name }	Creates a new organization.
GET	/api/organizations	None	Retrieves all organizations the logged-in user belongs to.
GET	/api/organizations/{org_id}	None	Fetches details for a specific organization.

POST	/api/organizations/{org_id}/members	{ user_id, role }	Adds an existing user to the organization.
-------------	-------------------------------------	-------------------	--

3. Teams (/api/teams) (New)

Teams are nested under organizations. Creating them happens at the org level, but managing them directly uses the [team_id](#).

Method	Endpoint	Request Payload	Description
POST	/api/organizations/{org_id}/teams	{ name }	Creates a new team inside an organization.
GET	/api/organizations/{org_id}/teams	<i>None</i>	Lists all teams within an organization.
GET	/api/teams/{team_id}	<i>None</i>	Gets the specific details of a single team.
POST	/api/teams/{team_id}/members	{ user_id, role }	Assigns a user to a specific team (e.g., 'LEAD' or 'MEMBER').

4. Projects (/api/projects) (Updated)

Projects are now correctly nested under their specific [teams](#).

Method	Endpoint	Request Payload	Description
--------	----------	-----------------	-------------

POST	/api/teams/{team_id}/projects	{ name, repo_link, meeting_link, storage_link }	Creates a new project assigned to that team.
GET	/api/teams/{team_id}/projects	<i>None</i>	Lists all projects owned by the team.
GET	/api/projects/{project_id}	<i>None</i>	Gets the specific project details and resource links.
PATCH	/api/projects/{project_id}	{ repo_link, meeting_link, storage_link }	Updates specific resource links (e.g., adding a Figma link later).
DELETE	/api/projects/{project_id}	<i>None</i>	Deletes the project.

With this spec, your backend is fully mapped out for Phase 1.

Do you want to write the raw SQL **CREATE TABLE** scripts next to run in your PostgreSQL database, or do you want to start writing the Spring Boot **JdbcTemplate** repository for the **users** table?

Phase 2

Phase 2: Issue Tracking & Milestones

With the workspace foundation laid, this phase builds the actual task management engine.

- **Milestone Management:** Creating specific milestones (e.g., "Mid-Semester Demo") that act as containers for tasks.
- **Issue Creation:** Allowing users to create simple tasks or bugs with titles and descriptions.
- **State Machine:** Implementing the minimal lifecycle tracking so issues can move between "To-Do", "In Progress", and "Done".
- **Task Assignment:** Linking tasks to specific `user_ids` from your Phase 1 tables.
- **Progress & Blocker Logging:** A system to record updates, specifically logging blockers like sickness or exact days delayed.

Database Architecture

To keep your raw SQL queries fast and clean, we will use straightforward foreign keys referencing the tables from Phase 1.

1. `milestones` Table

Groups tasks together to calculate the progress bar updates.

Column	Type	Constraints	Description
<code>id</code>	UUID/SERIAL	PRIMARY KEY	Unique identifier.
<code>project_id</code>	UUID/INT	FOREIGN KEY (<code>projects.id</code>)	The project this belongs to.
<code>name</code>	VARCHAR(150)	NOT NULL	E.g., "MVP Release".

due_date	DATE	NULLABLE	Target deadline for the milestone.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time.

2. issues Table

The core tracker for the work.

Column	Type	Constraints	Description
id	UUID/SERIAL	PRIMARY KEY	Unique identifier.
project_id	UUID/INT	FOREIGN KEY (projects.id)	Links to the parent project.
milestone_id	UUID/INT	FOREIGN KEY (milestones.id)	NULLABLE. Groups issue into a milestone.
assignee_id	UUID/INT	FOREIGN KEY (users.id)	NULLABLE. The team member responsible.
type	VARCHAR(50)	NOT NULL	'BUG', 'TASK', 'FEATURE'.
title	VARCHAR(255)	NOT NULL	Issue title.

description	TEXT	NULLABLE	Detailed breakdown.
status	VARCHAR(50)	DEFAULT 'TO_DO'	Tracks state: 'TO_DO', 'IN_PROGRESS', 'DONE'.

3. progress_logs Table (Comments & Delays)

Handles the teacher's requirement for tracking sick days and blockers.

Column	Type	Constraints	Description
id	UUID/SERIAL	PRIMARY KEY	Unique identifier.
issue_id	UUID/INT	FOREIGN KEY (issues.id)	The specific task being updated.
user_id	UUID/INT	FOREIGN KEY (users.id)	Who posted the update.
comment	TEXT	NOT NULL	The update text (e.g., "Sick today").

delay_days	INT	DEFAULT 0	Specific numerical delay tracker.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the comment was posted.

Note on the Progress Bar (Raw SQL Approach)

Since you prefer maintaining control with raw SQL, you do not need to store the "Progress Percentage" as a column in the database. Instead, you can calculate it dynamically on the backend whenever the frontend requests the milestone data.

You can execute a query like this in your Spring Boot repository:

```
SQL
SELECT
  COUNT(*) AS total_tasks,
  COUNT(CASE WHEN status = 'DONE' THEN 1 END) AS completed_tasks
FROM issues
WHERE milestone_id = ?;
```

Your backend can then simply return $(\text{completed_tasks} / \text{total_tasks}) * 100$ to the React frontend to render the progress bar.

Do you want to map out the exact REST API endpoints (like [POST /api/issues](#) or [GET /api/milestones/{id}/progress](#)) for this phase, or would you prefer to move straight into planning the React frontend component structure?